



Intrusion Detection & Prevention

A Complete Beginner's Guide

Open Source

Real-Time Detection

Rule-Based Engine

Cross-Platform

Topics: What is Snort • Architecture • Modes • Installation • Configuration • Rules • Commands • Use Cases • Best Practices

01 — What is Snort?

Snort is a **free, open-source** Network Intrusion Detection and Prevention System (NIDS/NIPS) originally developed by Martin Roesch in 1998 and now maintained by Cisco. It monitors network traffic in real time, analyzing packets against a library of predefined rules to identify suspicious or malicious behaviour.

Think of Snort as a **network watchdog** — it silently inspects every packet crossing your network and raises an alarm the moment it recognises something dangerous.

WHY SNORT?

- ✓ 100% Free & Open Source — no licensing cost
- ✓ Massive community with 1M+ deployments worldwide
- ✓ Regular rule updates for emerging threats
- ✓ Flexible — sniffer, logger, and full IDS/IPS in one
- ✓ Runs on Linux, Windows, macOS, BSD
- ✓ Integrates with SIEM tools (Splunk, Elastic, etc.)

IDS vs IPS — Key Difference

Feature	IDS (Detection)	IPS (Prevention)
Action	Alerts only	Blocks + alerts
Placement	Passive / out-of-band	Inline / in-line
Risk	Cannot stop attack	May block legit traffic
Snort mode	Sniff / Logger mode	Inline IPS mode

02 — Snort Architecture

Snort operates through a pipeline of four major components. Understanding this pipeline is essential to configuring and tuning Snort effectively.

1

Packet Decoder
Captures raw packets from the network interface (NIC) and decodes them by protocol layer — Ethernet → IP → TCP/UDP/ICMP → payload.

2

Preprocessors
Reassemble fragmented packets, normalize traffic (HTTP, FTP, DNS), and detect anomalies before rules are applied. Acts as a first-pass filter.

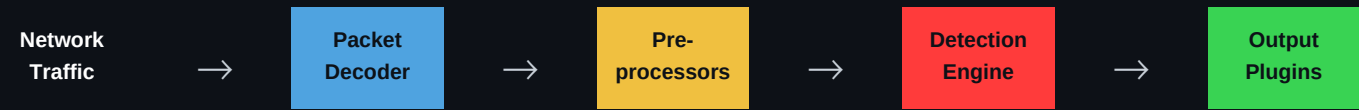
3

Detection Engine
The core of Snort. Compares decoded packets against all loaded rules. Uses a pattern-matching algorithm (Aho-Corasick) for speed.

4

Output Plugins
Delivers results: console alerts, log files, database writes, syslog, PCAP dumps, or external SIEM integration.

Data Flow Diagram



03 — Three Operating Modes

■ Sniffer Mode

Simply reads packets off the wire and displays them on screen — similar to tcpdump or Wireshark. Great for understanding your network traffic.

```
snort -v # verbose — show IP/TCP headers
snort -vd # include data payload
snort -vde # also show link-layer headers
snort -v -i eth0 # specify network interface
```

■ Packet Logger Mode

Captures and saves packets to disk for later analysis. Useful for forensic investigations or capturing traffic for offline review.

```
snort -dev -l /var/log/snort
# Save to directory
snort -dev -l /var/log/snort -h 192.168.1.0/24
# Log only your home network traffic
snort -dv -r packet.log # Read back saved log
```

■ IDS / IPS Mode

The primary production mode. Snort loads your rules file and monitors live traffic, generating alerts for rule matches. In inline mode, it also blocks malicious packets.

```
snort -dev -l /var/log/snort -h 192.168.1.0/24 \
-c /etc/snort/snort.conf
# -c specifies the configuration file
snort -A console -q -c /etc/snort/snort.conf -i eth0
# -A console: print alerts to screen
```

04 — Installation

Linux (Ubuntu / Debian)

```
# Step 1 – Update package lists
sudo apt update && sudo apt upgrade -y

# Step 2 – Install Snort
sudo apt install snort -y

# During install: enter your HOME_NET (e.g. 192.168.1.0/24)

# Step 3 – Verify installation
snort --version

# Step 4 – Install from source (for latest Snort 3)
sudo apt install -y build-essential libpcap-dev libpcrc3-dev \
libdumbnet-dev bison flex zlib1g-dev
```

Snort 3 (Latest) from Source

```
# Install libdaq first
git clone https://github.com/snort3/libdaq.git
cd libdaq && ./bootstrap && ./configure && make && sudo make install

# Build Snort 3
git clone https://github.com/snort3/snort3.git
cd snort3 && ./configure_cmake.sh --prefix=/usr/local
cd build && make -j$(nproc) && sudo make install

# Verify
snort --version
```

Windows Installation

Windows Setup Steps

- Download Snort installer from **snort.org** (choose Win32 package)
- Install **WinPcap** or **Npcap** (packet capture library)
- Run the Snort installer and follow the wizard
- Add Snort to your **PATH** environment variable
- Open cmd as Administrator and run: `snort --version`
- Download community rules from snort.org and extract to **C:\Snort\rules**

05 — Configuration File (snort.conf)

The **snort.conf** file (located at `/etc/snort/snort.conf`) is the brain of Snort's configuration. It is divided into several sections, each controlling a different aspect of Snort's behaviour.

Step 1 — Network Variables

```
var HOME_NET 192.168.1.0/24 # Your protected network
var EXTERNAL_NET !$HOME_NET # Everything else
var HTTP_SERVERS $HOME_NET
var DNS_SERVERS $HOME_NET
var HTTP_PORTS 80,8080,443
var SSH_PORTS 22
```

Step 2 — Decoder Configuration

```
# Enable/disable specific decoder alerts
config disable_decode_alerts
config enable_decode_drops
```

Step 3 — Preprocessors

```
# Normalise HTTP traffic
preprocessor http_inspect: global iis_unicode_map unicode.map 1252
preprocessor http_inspect_server: server default \
http_methods { GET POST PUT } ports { 80 8080 }
# Detect port scans
preprocessor sfportscan: proto { all } memcap { 10000000 }
```

Step 4 — Output Plugins

```
# Log alerts to unified2 format (for Barnyard2 / SIEM)
output unified2: filename snort.log, limit 128
# Console output
output alert_fast: stdout
# Syslog
output alert_syslog: LOG_AUTH LOG_ALERT
```

Step 5 — Include Rule Files

```
# Include all community rules
include $RULE_PATH/local.rules
include $RULE_PATH/community.rules
include $RULE_PATH/emerging-threats.rules
```

06 — Writing Snort Rules

Snort rules are the heart of the detection engine. Each rule is a single line telling Snort **what to look for** and **what action to take**. Rules are stored in `.rules` files and loaded via `snort.conf`.

Anatomy of a Snort Rule

alert	tcp	\$EXTERNAL_NET any	→	\$HOME_NET 80	(msg:"Test"; sid:1001;)
ACTION	PROTOCOL	SOURCE IP:PORT	DIR	DEST IP:PORT	OPTIONS

Rule Actions

Action	Description	Use Case
alert	Generate alert + log packet	Primary detection action
log	Log packet only, no alert	Passive traffic capture
pass	Ignore the packet	Whitelist trusted traffic
drop	Block + log (inline IPS only)	Active prevention
reject	Block + send TCP RST/ICMP error	Inline IPS with notification
sdrop	Block silently, no log	Silent blocking

Supported Protocols

tcp Transmission Control Protocol	udp User Datagram Protocol	icmp Internet Control Message	ip All IP traffic
-----------------------------------	----------------------------	-------------------------------	-------------------

06b — Rule Options Reference

Rule options appear inside parentheses after the rule header. They are separated by semicolons and provide precise matching criteria. Options fall into four categories: General, Payload, Non-Payload, and Post-Detection.

Keyword	Category	Description & Example
<code>msg</code>	General	Human-readable alert message: <code>msg:"SQL Injection Attempt";</code>
<code>sid</code>	General	Unique rule ID. Use >1,000,000 for custom rules: <code>sid:1000001;</code>
<code>rev</code>	General	Rule revision number: <code>rev:1;</code>
<code>classtype</code>	General	Rule category: <code>classtype:attempted-recon;</code>
<code>priority</code>	General	Alert priority 1–10 (1=highest): <code>priority:1;</code>
<code>content</code>	Payload	Match string in payload: <code>content:"GET /admin"; content:" 90 90 90 ";</code>
<code>nocase</code>	Payload	Case-insensitive content match: <code>content:"password"; nocase;</code>
<code>offset</code>	Payload	Start content search at byte N: <code>content:"GET"; offset:0;</code>
<code>depth</code>	Payload	Limit search to first N bytes: <code>depth:50;</code>
<code>pcre</code>	Payload	Perl-compatible regex: <code>pcre:"/select.+from/i";</code>
<code>flow</code>	Non-Payload	Traffic direction: <code>flow:to_server,established;</code>
<code>flags</code>	Non-Payload	TCP flags: <code>flags:S; (SYN) flags:SA; (SYN-ACK)</code>
<code>ttl</code>	Non-Payload	IP TTL value: <code>ttl:<10;</code>
<code>dsize</code>	Non-Payload	Payload size in bytes: <code>dsize:>512;</code>
<code>threshold</code>	Post-Detect	Rate limit alerts: <code>threshold: type limit, track by_src, count 5, seconds 60;</code>
<code>react</code>	Post-Detect	Active response (block/warn): <code>react: block;</code>
<code>reference</code>	General	Link to CVE/URL: <code>reference:cve,2021-44228;</code>

07 — Practical Rule Examples

■ Detect ICMP Ping (Network Recon)

```
alert icmp any any -> $HOME_NET any (msg:"ICMP Ping Detected"; itype:8; sid:1000001; rev:1;)
```

■ Detect Telnet Connection (Insecure Protocol)

```
alert tcp any any -> $HOME_NET 23 (msg:"Telnet Connection Attempt"; flow:to_server,established; sid:1000002; rev:1;)
```

■ Detect SSH Brute Force (5+ attempts / minute)

```
alert tcp $EXTERNAL_NET any -> $HOME_NET 22 (msg:"SSH Brute Force"; flow:to_server; flags:S; threshold:type threshold, track by_src, count 5, seconds 60; sid:1000003; rev:1;)
```

■ Detect SQL Injection Attempt

```
alert tcp $EXTERNAL_NET any -> $HTTP_SERVERS $HTTP_PORTS (msg:"SQL Injection - UNION SELECT"; flow:to_server,established; content:"UNION"; nocase; content:"SELECT"; nocase; pcre:"/UNION.+SELECT/i"; sid:1000004; rev:1;)
```

■ Detect FTP Login Attempt

```
alert tcp any any -> $HOME_NET 21 (msg:"FTP Login Attempt"; flow:to_server,established; content:"USER"; nocase; sid:1000005; rev:1;)
```

■ Detect Port Scan (many SYN packets)

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"Possible Port Scan"; flags:S; threshold:type threshold, track by_src, count 20, seconds 5; sid:1000006; rev:1;)
```

■ Detect HTTP GET to /etc/passwd (LFI)

```
alert tcp $EXTERNAL_NET any -> $HTTP_SERVERS $HTTP_PORTS (msg:"LFI Attack - /etc/passwd"; flow:to_server,established; content:"GET"; http_method; content:"/etc/passwd"; http_uri; nocase; sid:1000007; rev:1;)
```

■ Detect DNS Query to Suspicious Domain

```
alert udp $HOME_NET any -> any 53 (msg:"DNS Query - Suspicious TLD"; content:"|00 01 00 00|"; offset:4; depth:4; sid:1000008; rev:1;)
```

08 — Essential Command Reference

Test Configuration File	Run in IDS Mode (console alerts)
<pre>snort -T -c /etc/snort/snort.conf # -T = test mode (validates config without capturing)</pre>	<pre>sudo snort -A console -q -c /etc/snort/snort.conf -i eth0 # -A console: alerts to stdout -q: quiet mode # -i eth0: capture from interface eth0</pre>
Run with Verbose Output	Log to Directory
<pre>sudo snort -dev -c /etc/snort/snort.conf -i eth0 # -d: dump payload -e: link-layer headers</pre>	<pre>sudo snort -c /etc/snort/snort.conf -i eth0 -l /var/log/snort</pre>
Read from PCAP File (offline analysis)	Check Snort Version
<pre>snort -c /etc/snort/snort.conf -r capture.pcap</pre>	<pre>snort --version snort -V</pre>
Run Specific Rules File Only	List Available Interfaces
<pre>sudo snort -A console -q -i eth0 -c /etc/snort/snort.conf \ --rule-path /etc/snort/rules/local.rules</pre>	<pre>snort --daq-list ip link show</pre>

09 — Reading Snort Alerts

When Snort triggers a rule, it outputs an alert. Understanding the alert format is critical for incident response.

Sample Alert Output

```
[**] [1:1000004:1] SQL Injection - UNION SELECT [**]
[Classification: Web Application Attack] [Priority: 1]
06/05-14:32:17.482931 203.0.113.42:54321 -> 192.168.1.10:80
TCP TTL:64 TOS:0x0 ID:12345 IpLen:20 DgmLen:512 DF
***AP*** Seq: 0x1A2B3C4D Ack: 0x5E6F7A8B Win: 0x2000 TcpLen: 32
```

Alert Component	Meaning
[1:1000004:1]	Generator ID : Rule SID : Revision number
SQL Injection - UNION SELECT	The msg field from your rule
[Priority: 1]	Alert priority (1=critical, 4=informational)
06/05-14:32:17.482931	Timestamp: Month/Day-HH:MM:SS.microseconds
203.0.113.42:54321	Source IP address and port number
192.168.1.10:80	Destination IP address and port number
TCP TTL:64	Protocol, Time-To-Live value of packet
AP	TCP flags: A=ACK, P=PUSH (SYN=S, FIN=F, RST=R)

Log File Locations

Default Log Paths

- `/var/log/snort/alert` — Plain text alert log
- `/var/log/snort/snort.log.*` — Binary PCAP packet log
- `/var/log/snort/appid/` — Application ID logs (Snort 3)
- Use `u2spewfoo snort.log.*` to read unified2 binary logs
- Use `tcpdump -r snort.log.*` to open packet captures

10 — Common Use Cases

■ Network Perimeter Monitoring

Deploy Snort at the network edge (between firewall and internal network) to detect inbound attacks — port scans, exploit attempts, malware callbacks.

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"Inbound Port Scan"; flags:S; threshold:type threshold, track by_src, count 30, seconds 10; sid:2000001; rev:1;)
```

■ Web Application Protection

Monitor HTTP traffic to detect common web attacks: SQL injection, XSS, LFI, RFI, and command injection attempts.

```
alert tcp any any -> $HTTP_SERVERS $HTTP_PORTS (msg:"XSS Attempt"; flow:to_server; content:"<script>"; nocase; http_uri; sid:2000002; rev:1;)
```

■ Malware C2 Detection

Detect malware communicating with command-and-control servers by looking for known beacon patterns or suspicious DNS queries.

```
alert udp $HOME_NET any -> any 53 (msg:"Possible DNS Tunneling"; dsize:>200; sid:2000003; rev:1;)
```

■ Compliance + Audit Logging

Log all traffic to/from sensitive servers (PCI-DSS, HIPAA) for compliance purposes. Use packet logger mode for full PCAP.

```
snort -dev -l /var/log/snort -h 10.0.0.0/24 -c /etc/snort/snort.conf
```

11 — Best Practices

■ Rule Management

- Always test new rules with `snort -T` before deploying
- Use SID numbers above 1,000,000 for custom rules to avoid conflicts
- Set the `rev` option and increment it every time you modify a rule
- Comment your rules — explain what they detect and why
- Regularly update community rules from [snort.org](https://www.snort.org) or [GitHub](https://github.com/SecWiki/snort-rules)

■ Reducing False Positives

- Use the `threshold` keyword to suppress noisy rules
- Add `flow` keywords to restrict rule direction (established sessions only)
- Use `pass` rules to whitelist trusted internal scanners or monitoring tools
- Enable `-Q` (quiet mode) in production to suppress routine output
- Tune `HOME_NET` and `EXTERNAL_NET` variables accurately

■ Security & Performance

- Run Snort as a non-root user where possible (use `cap` for capabilities)
- Use **Barnyard2** or **Snorby** to offload alert processing
- Monitor CPU/RAM — the detection engine is CPU-intensive
- Use `PF_RING` or `DPDK` for high-speed (10Gbps+) network interfaces
- Archive and rotate log files — `snort.log` files grow quickly

■ Integration Tips

- Connect Snort alerts to **Elasticsearch/Kibana (ELK)** for dashboards
- Use **Splunk** with the Snort add-on for enterprise SIEM correlation
- Pair with **pfSense** or **OPNsense** for an all-in-one firewall + IDS
- Consider **Suricata** as a Snort-compatible alternative with multi-threading
- Use **PulledPork** script to automatically update and manage rules

12 — Quick Reference Cheat Sheet

Command	Purpose
<code>snort -V</code>	Show version information
<code>snort -v -i eth0</code>	Sniffer mode on eth0
<code>snort -vd -i eth0</code>	Sniff with payload data
<code>snort -T -c snort.conf</code>	Test configuration file
<code>snort -A console -q -c snort.conf -i eth0</code>	IDS mode, console alerts
<code>snort -c snort.conf -r file.pcap</code>	Analyse offline PCAP
<code>snort -c snort.conf -l /var/log/snort</code>	Log to directory
<code>snort -dev -l /var/log/snort -h 192.168.1.0/24</code>	Packet logger mode
<code>snort --daq-list</code>	List capture modules

Rule Template

```
alert <protocol> <src_ip> <src_port> -> <dst_ip> <dst_port> (  
  msg:"<Description of what this rule detects>";  
  flow:<direction>;  
  content:"<string to match>"; [nocase;] [offset:N;] [depth:N;]  
  [pcre:"<regex>/flags";]  
  [threshold:type <limit|threshold|both>, track by_<src|dst>,  
  count <N>, seconds <T>;]  
  classtype:<category>;  
  sid:<unique_number>;  
  rev:<revision>;  
)
```

Key snort.conf Variables

Variable	Default Value	Meaning
<code>\$HOME_NET</code>	192.168.1.0/24	Your internal protected network
<code>\$EXTERNAL_NET</code>	!\$HOME_NET	Everything outside HOME_NET
<code>\$HTTP_SERVERS</code>	\$HOME_NET	Your web servers
<code>\$HTTP_PORTS</code>	80,8080,443	Web server ports
<code>\$RULE_PATH</code>	/etc/snort/rules	Directory containing .rules files
<code>\$LOG_PATH</code>	/var/log/snort	Alert and log output directory

13 — Troubleshooting Common Issues

Snort not detecting traffic

- Ensure correct interface: use `ip link show` to list NICs
- Run with `sudo` — Snort needs root for raw packet capture
- Verify `HOME_NET` is correctly set to your actual network range
- Test with: `snort -v -i eth0` to confirm packets are being read

Too many false positives

- Add threshold to noisy rules to rate-limit alerts
- Use pass rules to whitelist known-good traffic sources
- Restrict rules with `flow:established` to avoid SYN-only matches
- Disable specific community rules that don't apply to your environment

Configuration test fails

- Run `snort -T -c /etc/snort/snort.conf` and read error output
- Check that all include paths in `snort.conf` are correct and files exist
- Verify that variables like `$RULE_PATH` point to existing directories
- Check syntax of custom rules — missing semicolons are a common cause

High CPU / dropped packets

- Reduce active rule count — remove rules irrelevant to your environment
- Use `PF_RING` or `DPDK` for high-throughput networks (>1Gbps)
- Use `Barnyard2` to handle output separately from the detection engine
- Consider Snort 3 which has better multi-threading than Snort 2.x

■ Further Learning & Resources

Resource	URL / Details
Official Snort Website	https://www.snort.org
Snort 3 Rule Writing Guide	https://docs.snort.org/rules/
Community Rules (free)	https://www.snort.org/downloads#rules
Emerging Threats Rules	https://rules.emergingthreats.net/
TryHackMe — Snort Room	https://tryhackme.com/room/snort
Snort GitHub Repository	https://github.com/snort3/snort3
PulledPork (rule updater)	https://github.com/shirkgod/pulledpork

■ **Snort for Beginners** — Open Source Network IDS/IPS • Real-time Packet Analysis • Rule-Based Threat Detection